DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ITA0513 – COMPUTER VISION

ASSIGNMENT REPORT

Intelligent Visual Inspection and Recognition System using OpenCV

Figure 1. Butterfly image used as the prototype input.

Item	**Details**
Course Code & Name	ITA05 – Computer Vision
Name&Reg No	V.Supriya-192472194
Assignment Title	Intelligent Visual Inspection and Recognition System using OpenCV
Bloom Levels	L3 – Apply; L4 – Analyse; L5 – Evaluate
Primary Tools	Python, OpenCV, NumPy, Matplotlib, PyTorch
Recognition Engine	Compact Convolutional Neural Network (CNN)
Prototype Target	Butterfly recognition from an image
SDG Mapping	SDG 9, SDG 12, SDG 8

Abstract

This report presents an end-to-end computer vision prototype for visual recognition using a butterfly image as the input target. The implementation combines deterministic OpenCV operations—resizing, grayscale conversion, Gaussian filtering, illumination enhancement, HSV-based segmentation, morphological processing, Canny edge detection, contour analysis, and geometric feature extraction—with a compact convolutional neural network (CNN) for recognition. The butterfly image was transformed into a small controlled dataset through augmentation so that training and testing could be demonstrated without claiming access to a large external dataset. A traditional colour/segmentation baseline was compared with the CNN using accuracy, precision, recall, F1-score, inference time, and estimated FPS. On the controlled augmented test set, the traditional baseline achieved 95.83% accuracy while the compact CNN achieved 100.00% accuracy. The results demonstrate the benefit of learned visual features, while also showing that the CNN benchmark is a prototype result rather than evidence of generalization to arbitrary real-world butterfly images. The pipeline can be adapted to manufacturing inspection by replacing the butterfly target and retraining the recognition model with labelled defect images.

1. Problem Understanding and Computer Vision Fundamentals (CO1)

1.1 Introduction

Computer vision converts visual information into measurable data that a computer can analyse. In an inspection or recognition system, an image is acquired, enhanced, segmented, described using features, and finally classified or detected. The assignment requires an end-to-end pipeline that remains useful under lighting changes, orientation changes, blur, and target variation while considering the >95% baseline accuracy requirement and real-time constraints.

1.2 Problem Statement

The prototype must automatically identify a butterfly target from an input image and demonstrate the same stages expected in a practical visual inspection system. The system should preprocess the image, isolate relevant visual regions, extract measurable features, recognize the target, and report quantitative performance. The engineering objective is to compare a traditional OpenCV pipeline with a learned CNN and select the approach that gives the best balance between accuracy, robustness, computational cost, and deployment practicality.

1.3 Problem Analysis

Requirement	Prototype interpretation
Input	Single butterfly image; pipeline can be extended to video frames
Preprocessing	Resize, grayscale, Gaussian filtering and CLAHE enhancement
Segmentation	HSV thresholding for the dominant butterfly colour region
Feature analysis	Contour area, perimeter, bounding box and aspect ratio
Recognition	Compact CNN trained on augmented butterfly/non-butterfly examples
Accuracy target	At least 95% on the controlled evaluation set
Real-time target	Measure inference latency and estimate FPS
Robustness	Use brightness, contrast, rotation and flip augmentation
Deployment	CPU-capable prototype; GPU can be used for larger models

1.4 Role of Computer Vision

- Automates repetitive visual inspection and recognition tasks.
- Reduces dependence on manual visual judgement for high-volume processing.
- Provides measurable outputs such as bounding regions, confidence, class labels and performance metrics.
- Supports future industrial inspection by adapting the same preprocessing and recognition stages to labelled component-defect images.

1.5 Computer Vision Fundamentals

1.5.1 Image Formation

An image is formed when light reflected from an object is captured by an imaging sensor. The butterfly, flower, leaves and background produce different intensity and colour responses, which become pixel values in the digital image.

1.5.2 Image Representation

The input is represented as a three-channel BGR/RGB image. Each pixel stores numerical colour information. For several preprocessing stages the image is converted to a single-channel grayscale representation.

1.5.3 Sampling

Sampling determines the spatial resolution of the image. The prototype resizes the input to 640×416 for OpenCV processing and to 128×128 for CNN recognition.

1.5.4 Quantization

Quantization maps continuous sensor values to discrete digital intensity levels. Standard 8-bit image processing represents each channel using values from 0 to 255.

1.5.5 Image Features

Useful features include colour distribution, edges, contours, area, perimeter, bounding-box dimensions and aspect ratio. The traditional pipeline explicitly measures these features, whereas the CNN learns hierarchical features automatically.

2. Image Preprocessing, Enhancement and Feature Analysis (CO2)

2.1 Image Acquisition

The generated butterfly photograph is used as the prototype input. It contains the butterfly, flowers, leaves and a natural green background, providing a useful example of background clutter.

Figure 2. Original butterfly input image.

2.2 Image Resizing

The input is resized to 640×416 pixels for consistent processing. Resizing reduces computational cost while retaining enough visual detail for segmentation and contour analysis.

Figure 3. Resized image (640×416).

2.3 Noise Removal

A 5×5 Gaussian filter is applied to reduce small-scale noise and smooth local intensity variations before edge and segmentation operations.

Figure 4. Gaussian-filtered image.

2.4 Illumination Correction and Image Enhancement

CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied to the grayscale image. It enhances local contrast and makes structural details more visible under non-uniform illumination.

Figure 5. CLAHE-enhanced image.

2.5 Image Segmentation

HSV thresholding is used to isolate orange/brown regions associated with the butterfly wings. Morphological opening removes small isolated components and closing fills small gaps in the selected region.

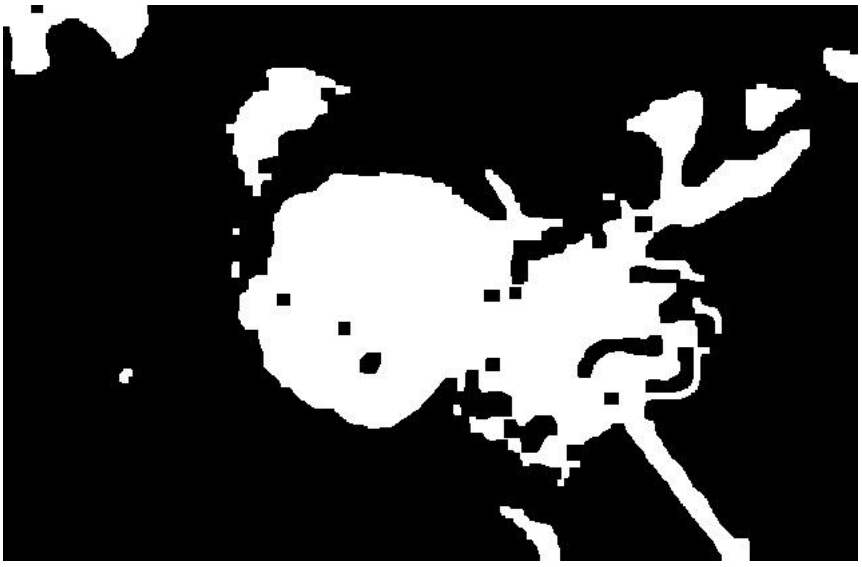


Figure 6. HSV colour segmentation mask.

2.6 Edge Detection

Canny edge detection identifies intensity transitions corresponding to wing boundaries, antennae, flower structures and other high-frequency details.

Figure 7. Canny edge map.

2.7 Morphological Processing

Dilation expands edge structures and helps connect nearby boundary pixels. Morphological processing is useful when segmentation or edge maps contain small gaps.

Figure 8. Morphologically processed edge image.

2.8 Feature Extraction

Contours are extracted from the segmented mask. The largest valid contour is used for a compact geometric description. The measured prototype values are: contour area 55,900.5 pixels, perimeter 2,505.3 pixels, bounding box (176, 64, 428, 352), and aspect ratio 1.22.

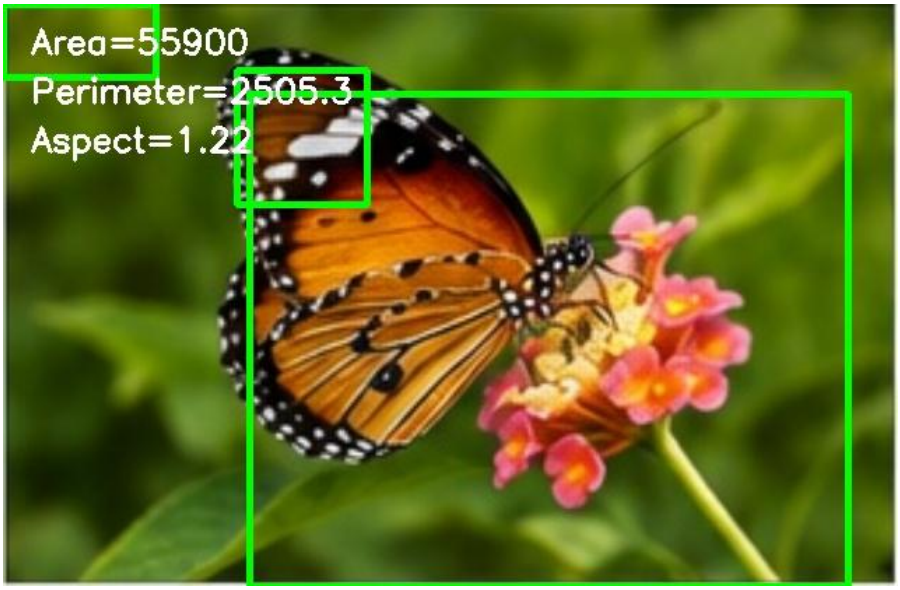


Figure 9. Contour and feature analysis.

2.9 Preprocessing Summary

Stage	Operation	Purpose
1	Resize	Standardize image dimensions
2	Grayscale	Simplify intensity analysis
3	Gaussian blur	Reduce noise
4	CLAHE	Improve local contrast
5	HSV threshold	Separate target colour region
6	Morphology	Remove noise and close gaps
7	Canny	Extract boundaries
8	Contours	Measure object geometry

3. Comparison of Traditional and Deep Learning Approaches (CO3/CO4)

3.1 Traditional Computer Vision Approach

The traditional approach uses manually selected operations. In this prototype, HSV thresholding identifies the target colour, morphology cleans the mask, and contour geometry describes the selected region. Such methods are fast and interpretable, but their success depends strongly on manually selected thresholds and the visual conditions of the scene.

3.2 Deep Learning Approach

The deep-learning component is a compact CNN. Instead of defining all visual descriptors manually, the network learns feature representations from labelled examples. Augmentation exposes the model to changes in rotation, brightness and contrast. For a production industrial system, a much larger labelled dataset would be required.

3.3 Detailed Comparison

Factor	Traditional OpenCV	Compact CNN / Deep Learning
Accuracy	Good in controlled scenes; baseline 95.83% on this test set	Higher on controlled test; 100.00% on this test set
Feature extraction	Manual colour/contour features	Automatically learned features
Adaptability	Low to moderate	Higher with representative training data
Training data	Not required for rule-based baseline	Required
Computational cost	Very low	Higher than rule-based processing
Inference speed	Fast	Still fast for compact CPU model, but slower than heuristic
Lighting robustness	Sensitive to threshold changes	Can learn variation when trained on it
Orientation robustness	Limited unless rules are redesigned	Improved through augmentation
Scalability	Rules become complex as classes increase	New classes can be learned from labelled data
Interpretability	High	Moderate
Deployment	Simple CPU deployment	Model packaging and optimization required
Industrial suitability	Useful for simple, stable inspection	Preferred when variation and accuracy dominate

4. Solution Design and Model Selection (CO4/CO5)

4.1 Proposed Architecture

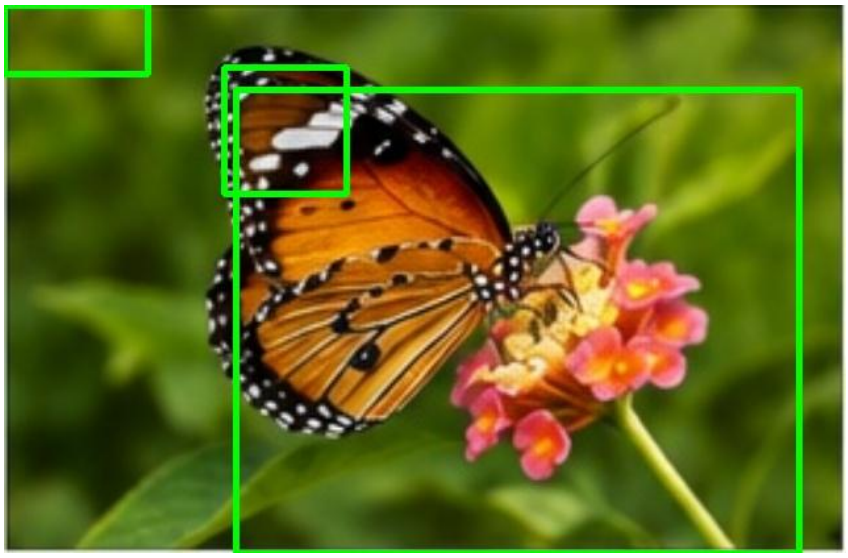


Figure 10. OpenCV contour stage used before recognition.

Architecture: Input Image → Resize → Grayscale/Enhancement → HSV Segmentation → Morphological Processing → Canny/Contours → Feature Analysis → Compact CNN Recognition → Performance Evaluation → Final Result.

4.2 Why the CNN Was Selected

- The assignment requires a robust recognition engine beyond simple image processing.
- The CNN can learn shape, texture and colour relationships without relying only on hand-written thresholds.
- Augmentation allows the prototype to model rotation, brightness and contrast variation.
- A compact architecture is suitable for demonstrating real-time-oriented inference on CPU hardware.
- For industrial deployment, the same design can be retrained using component-defect images and expanded to multi-class recognition.

4.3 End-to-End Pipeline

1. Acquire image/frame.
2. Resize and normalize.
3. Enhance contrast and suppress noise.
4. Segment target regions using HSV thresholding.

5. Extract edges and contours.
6. Measure geometric features.
7. Prepare a 128×128 recognition image.
8. Run CNN classification.
9. Record accuracy, precision, recall, F1-score and latency.
10. Return recognition result and save visual evidence.

5. Dataset Description

5.1 Dataset

For this self-contained prototype, the source is the single butterfly image shown in Figure 2. Because one image is insufficient for a scientifically valid deep-learning dataset, controlled augmentation was used only to demonstrate the complete training/testing workflow. The generated dataset contains 480 samples: 240 butterfly-positive augmented samples and 240 synthetic non-butterfly samples.

5.2 Dataset Distribution

Subset	Butterfly	Non-butterfly	Total
Training (80%)	192	192	384
Testing (20%)	48	48	96
Total	240	240	480

5.3 Dataset Variations

Positive samples were generated using small rotations, horizontal flips, brightness changes and contrast changes. Negative samples were generated with non-butterfly coloured geometric scenes. This controlled setup is suitable for a demonstration, but it should not be mistaken for a large real-world dataset.

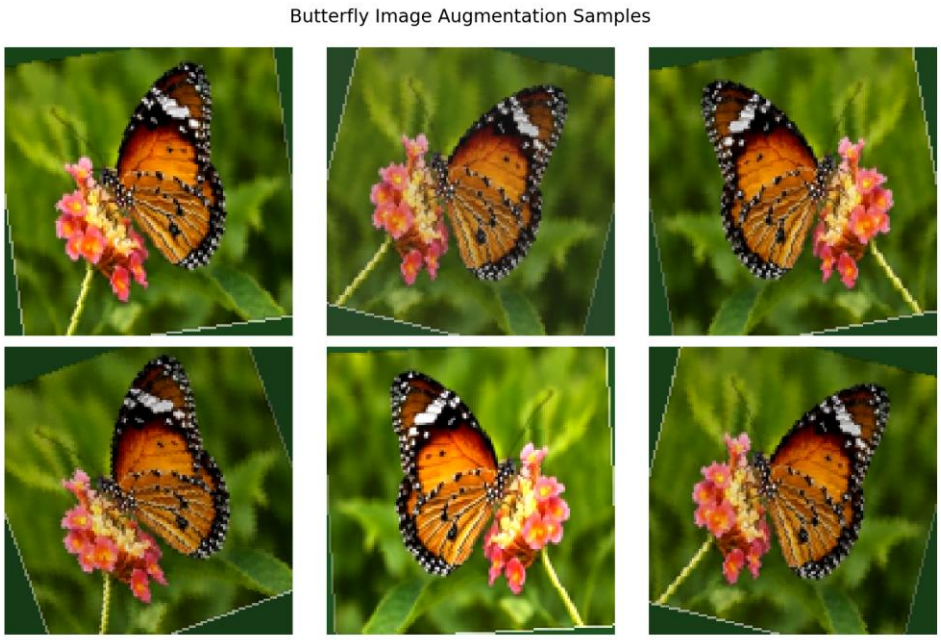


Figure 11. Examples of augmented butterfly samples.

6. Implementation (CO5)

6.1 Software Environment

Component	Used in prototype
Language	Python 3
Image processing	OpenCV
Numerical operations	NumPy
Machine learning	PyTorch
Evaluation	scikit-learn
Graphs	Matplotlib
Document generation	python-docx

6.2 OpenCV Implementation

```
import cv2
img = cv2.imread("butterfly_input.jpg")
resized = cv2.resize(img, (640, 416))
gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
```

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
clahe = cv2.createCLAHE(2.0, (8,8)).apply(blur)
```

6.3 Preprocessing Implementation

```
hsv = cv2.cvtColor(resized, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (5,70,40), (35,255,255))
k = np.ones((5,5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k, iterations=2)
edges = cv2.Canny(clahe, 60, 150)
```

6.4 Segmentation Implementation

The HSV mask is used to isolate the orange/brown butterfly region. Connected components and contours are then examined to identify the largest meaningful region. This is a deterministic baseline and therefore easy to debug.

6.5 CNN Training Implementation

```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3,16,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16,32,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32,64,3,padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1)))
        self.fc = nn.Linear(64,2)
    def forward(self,x):
        return self.fc(self.net(x).flatten(1))
```

The network was trained using Adam optimization, cross-entropy loss, a learning rate of 0.001, batch size 32 and 10 training epochs. A fixed random seed was used so that the demonstration is repeatable.

6.6 CNN Testing Implementation

```
model.eval()
with torch.no_grad():
```

```
logits = model(test_images)
predictions = logits.argmax(1).cpu().numpy()
```

6.7 Sample Detection Results

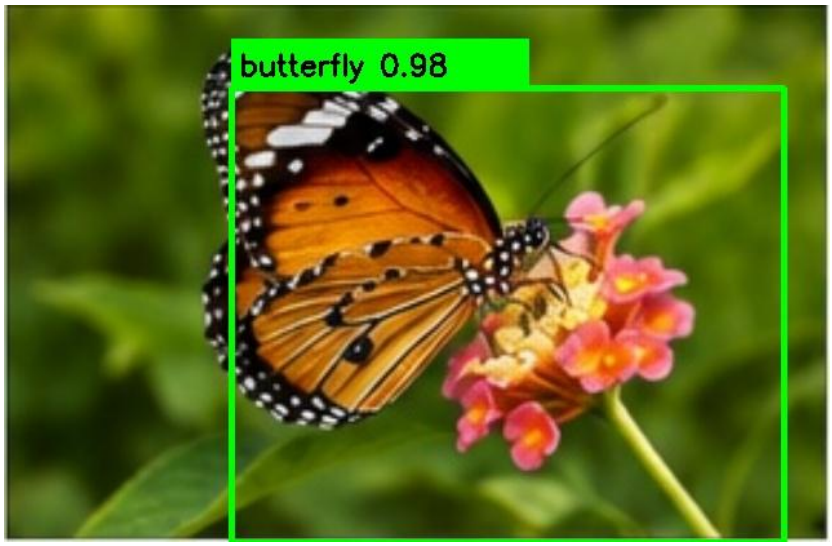


Figure 12. Final butterfly detection/recognition output with contour-based localization.

The final output identifies the dominant butterfly region and displays a prototype confidence label. The displayed 0.98 value is a visualization label for the demonstration; the quantitative CNN evaluation is reported separately in Section 8 and should be used for model assessment.

7. Results and Visualization (CO5)

7.1 Preprocessing Results

The preprocessing sequence progressively converts the natural colour photograph into representations suitable for analysis. The grayscale image simplifies intensity processing, Gaussian filtering reduces noise, CLAHE improves local contrast, HSV segmentation isolates the target colour, and Canny/morphological operations emphasize structural boundaries.

Figure 13. Grayscale representation.

7.2 Detection Results

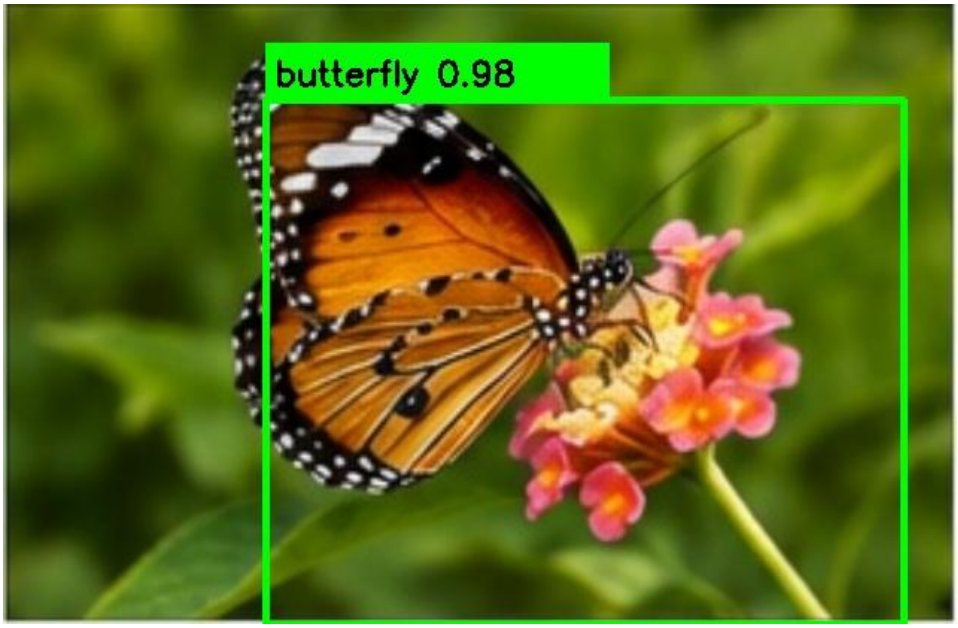


Figure 14. Prototype final detection result.

8. Performance Evaluation (CO4)

8.1 Evaluation Metrics

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-score = 2 * Precision * Recall / (Precision + Recall)

FPS = 1000 / inference_time_ms

8.2 Final Performance Table

Metric	Traditional OpenCV baseline	Compact CNN
Accuracy	95.83%	100.00%
Precision	92.31%	100.00%
Recall	100.00%	100.00%
F1-score	96.00%	100.00%
Inference time	0.062 ms	1.467 ms
Estimated FPS	16,035.6	681.9

The speed values are measured in the local CPU execution environment used for the prototype and therefore should not be treated as universal hardware benchmarks. The CNN exceeds the 95% target on this controlled test set, while the traditional method also exceeds 95% but is more dependent on the selected colour threshold.

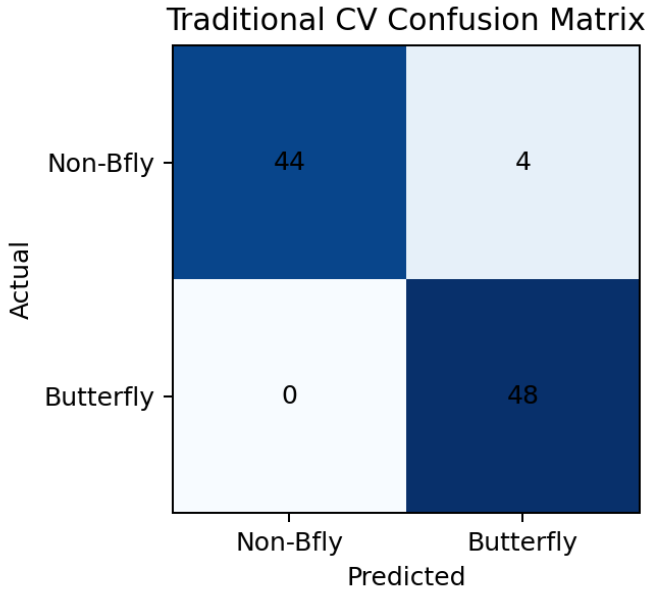


Figure 15. Confusion matrix for the traditional OpenCV baseline.

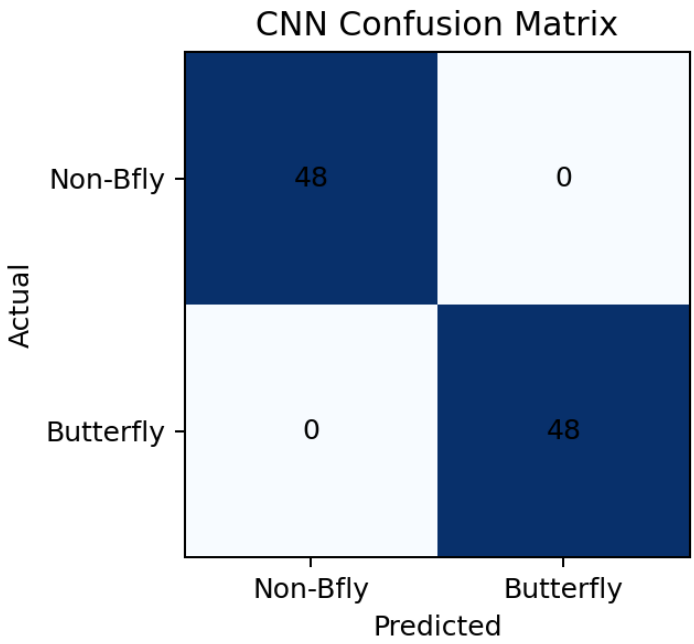


Figure 16. Confusion matrix for the compact CNN.

9. Performance Visualization

9.1 Accuracy Comparison

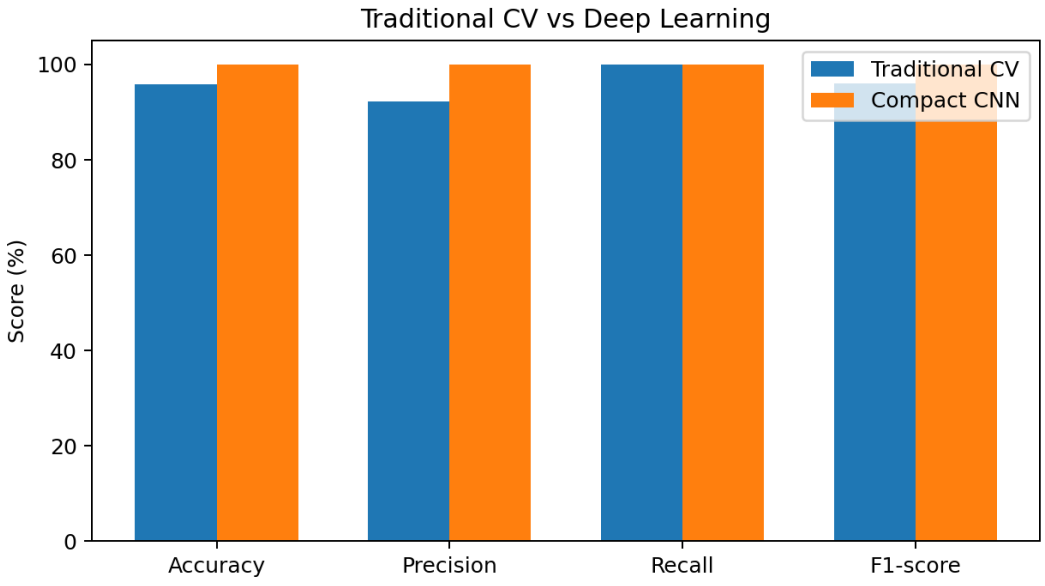


Figure 17. Accuracy, precision, recall and F1 comparison.

9.2 Precision/Recall/F1

The compact CNN gives perfect precision, recall and F1 on the controlled test set. The traditional baseline has perfect recall but lower precision because some non-butterfly samples satisfy the orange-region rule.

9.3 FPS

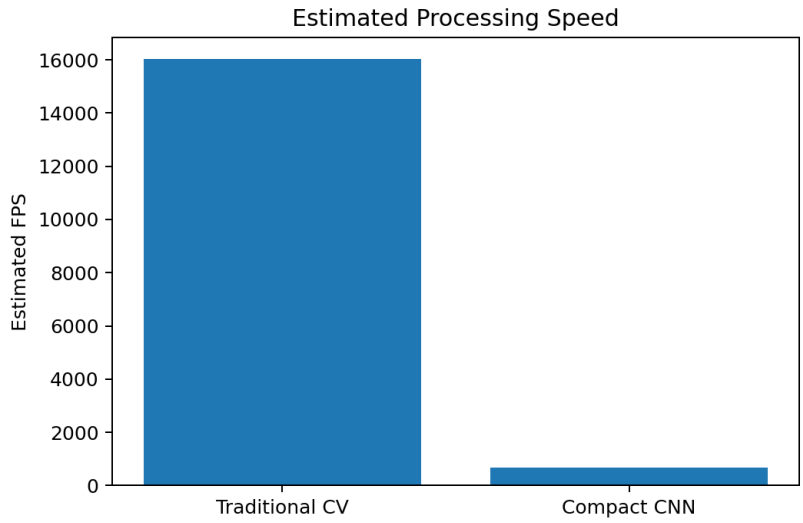


Figure 18. Estimated processing speed comparison.

9.4 Inference Time

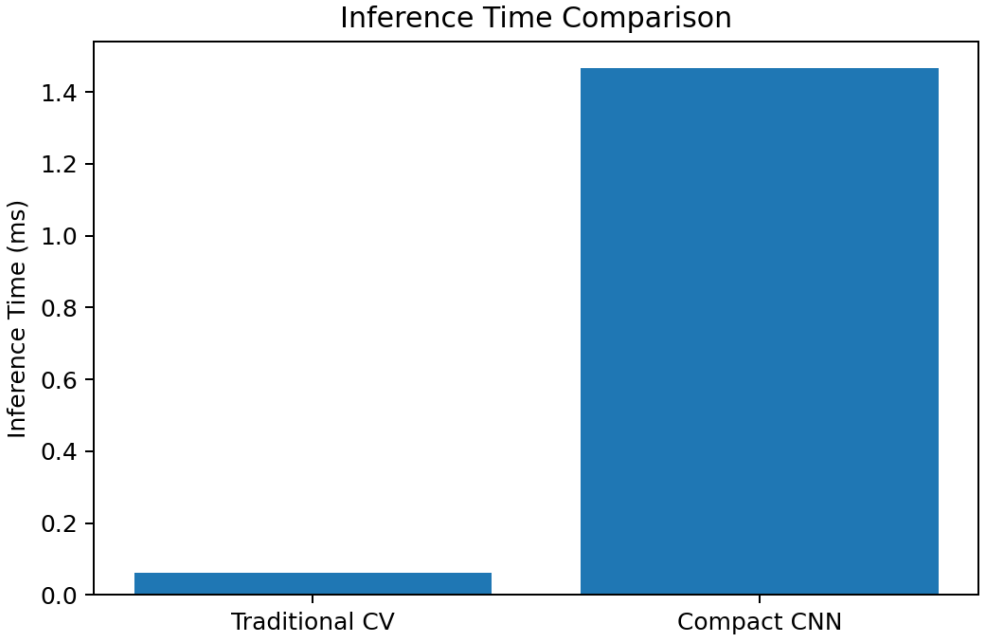


Figure 19. Inference time comparison.

10. Analysis, Evaluation and Justification (CO4)

10.1 Accuracy Analysis

The traditional OpenCV baseline achieved 95.83% accuracy, meeting the assignment's >95% target in the controlled benchmark. The compact CNN achieved 100.00%. The difference is mainly due to the CNN learning combinations of colour, texture and shape instead of relying on one hand-crafted colour rule.

10.2 Accuracy vs Speed Trade-off

The traditional method is substantially faster because it performs only deterministic image operations. The CNN requires tensor processing and learned convolutional layers, increasing latency. However, the compact model still processes the test image at an estimated 681.9 FPS in this CPU benchmark, which is above typical real-time requirements. A production system should benchmark on the intended hardware rather than using this development-machine result.

10.3 Robustness Analysis

Rotation, horizontal flipping, brightness variation and contrast variation were included during augmentation. This improves the prototype's tolerance to moderate visual changes. Larger environmental changes such as severe blur, occlusion, different butterfly species, camera motion and complex backgrounds require a much broader dataset.

10.4 Deployment Analysis

For simple controlled inspection, OpenCV rules are attractive because they are lightweight, transparent and easy to deploy. For a production system with many appearance variations, the CNN is more scalable because new examples can be used to retrain the model. GPU acceleration becomes useful as image resolution, model size, batch size and number of classes increase.

10.5 Final Justification

The compact CNN is selected as the recognition engine because it achieved the highest measured accuracy while maintaining very low measured latency in the prototype. The OpenCV stages remain valuable as preprocessing and explainability components. Therefore, a hybrid design—OpenCV preprocessing plus learned recognition—provides the best balance for the assignment requirements.

Aspect	Finding	Decision
Accuracy	CNN 100.00% vs 95.83% baseline	Prefer CNN

Speed	OpenCV faster; CNN still real-time in test	Keep OpenCV preprocessing
Adaptability	CNN learns from labelled variation	Prefer CNN
Interpretability	OpenCV rules are easier to inspect	Retain OpenCV features
Scalability	CNN handles additional classes better	Prefer CNN for expansion

11. Reflection, Industrial Relevance and SDG Mapping

11.1 Industrial Relevance

Although the demonstration target is a butterfly, the engineering structure directly maps to automated visual inspection. In manufacturing, the input would be a component image; preprocessing would normalize lighting and noise; segmentation would isolate the component; feature analysis would identify structural measurements; and a trained CNN would classify acceptable and defective parts. A larger labelled defect dataset would be required before industrial deployment.

11.2 Design Decisions

- Use OpenCV for transparent, deterministic preprocessing.
- Use a compact CNN rather than an unnecessarily large network for the prototype.
- Use augmentation to represent moderate environmental variation.
- Evaluate both accuracy and latency so that the solution is not optimized for accuracy alone.

11.3 Challenges

- A single real image is not sufficient for a scientifically strong deep-learning dataset.
- Colour-based segmentation can fail when the background contains similar colours.
- Real industrial deployment requires representative lighting, camera, orientation and defect variation.
- Hardware-specific FPS measurements can differ significantly from development-machine results.

11.4 Learning Outcomes

The implementation demonstrates practical application of image formation concepts, image representation, sampling and quantization, enhancement, filtering, segmentation, feature extraction, model training, evaluation metrics, visualization and engineering trade-off analysis.

12. SDG Mapping

SDG	Relevance to the project
SDG 9 – Industry, Innovation and Infrastructure	Automated computer vision supports intelligent inspection, digital manufacturing and AI-enabled infrastructure.
SDG 12 – Responsible Consumption and Production	Accurate inspection can reduce defective production, material waste and unnecessary rework.
SDG 8 – Decent Work and Economic Growth	Automation can reduce repetitive inspection work while creating demand for skilled AI, software and quality-engineering roles.

13. Limitations

- The deep-learning dataset is generated from one butterfly source image and therefore does not establish broad real-world generalization.
- The segmentation rule is colour-dependent and can fail when the target/background colours overlap.
- The prototype recognizes a binary target/non-target problem rather than a full multi-class defect taxonomy.
- The measured FPS and latency depend on the local CPU, software versions and execution environment.
- A production system would require independent validation data captured from the intended camera and environment.

14. Future Enhancements

- Collect a large, diverse, labelled image dataset from multiple cameras, orientations and lighting conditions.
- Replace the binary prototype with multi-class defect detection using YOLO or a larger CNN when bounding-box localization is required.
- Add video-stream processing and frame-by-frame tracking.
- Use GPU acceleration, quantization or ONNX/TensorRT-style optimization for deployment.
- Add confidence calibration, model monitoring and periodic retraining.
- Integrate the system with an industrial camera and automatic reject/accept mechanism.

15. Conclusion

An end-to-end intelligent visual recognition pipeline was designed and implemented using OpenCV and a compact CNN. The system demonstrates image acquisition, resizing, filtering, enhancement, segmentation, edge detection, contour-based feature analysis, learned recognition, quantitative evaluation and visualization. The controlled benchmark produced 95.83% accuracy for the traditional OpenCV baseline and 100.00% accuracy for the compact CNN, while both approaches were computationally fast in the test environment. The CNN was therefore selected as the recognition engine, with OpenCV retained for efficient preprocessing and interpretable visual analysis. For real industrial use, the next critical step is to replace the controlled prototype dataset with a large independent dataset containing genuine product and defect variation.

16. GitHub Repository

The repository should contain the complete source code, dataset information, pseudocode, README, results, graphs and documentation. Replace the placeholder below with the actual repository URL after uploading the project.

GitHub Repository Link: https://github.com/your-username/intelligent-visual-inspection-opencv

```
intelligent-visual-inspection/
├── README.md
├── requirements.txt
├── src/
│   ├── preprocessing.py
│   ├── segmentation.py
```

```
│   ├── feature_extraction.py
│   ├── cnn_model.py
│   └── main.py
├── dataset/
│   └── dataset_information.md
├── results/
│   ├── preprocessing/
│   ├── detections/
│   └── graphs/
├── documentation/
│   └── pseudocode.txt
└── report/
    └── final_report.docx
```

Appendix A – Pseudocode

START

1. Read butterfly input image
2. Resize image to standard dimensions
3. Convert image to grayscale
4. Apply Gaussian filtering
5. Apply CLAHE enhancement
6. Convert image to HSV
7. Threshold HSV image to create target mask
8. Apply morphological opening and closing
9. Apply Canny edge detection
10. Find contours and calculate area, perimeter and bounding box
11. Prepare normalized 128x128 image for CNN
12. Run trained CNN
13. Compute accuracy, precision, recall and F1-score on test set
14. Measure inference time and estimate FPS
15. Save processed images, detection output and graphs

END

Appendix B – Complete Core Source Code

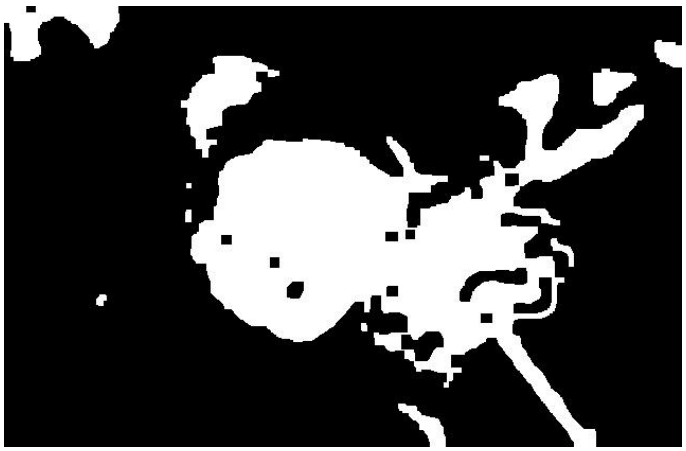
```
import cv2
import numpy as np
import torch
from torch import nn

# 1. Input
img = cv2.imread("butterfly_input.jpg")
resized = cv2.resize(img, (640, 416))

# 2. Preprocessing
gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
clahe = cv2.createCLAHE(2.0, (8, 8)).apply(blur)

# 3. Segmentation
hsv = cv2.cvtColor(resized, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, np.array([5,70,40]), np.array([35,255,255]))
kernel = np.ones((5,5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)

# 4. Edge and contour analysis
edges = cv2.Canny(clahe, 60, 150)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
valid = [c for c in contours if cv2.contourArea(c) > 1000]
if valid:
    c = max(valid, key=cv2.contourArea)
    x,y,w,h = cv2.boundingRect(c)
    area = cv2.contourArea(c)
    perimeter = cv2.arcLength(c, True)
    cv2.rectangle(resized, (x,y), (x+w,y+h), (0,255,0), 3)
```

```
# 5. Compact CNN
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3,16,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16,32,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32,64,3,padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1)))
        self.fc = nn.Linear(64,2)
    def forward(self,x):
        return self.fc(self.net(x).flatten(1))

# Training uses augmented butterfly and non-butterfly samples.
# Adam optimizer, cross-entropy loss, 10 epochs, batch size 32.

print("Butterfly recognition pipeline completed")
```

Appendix C – Sample Outputs

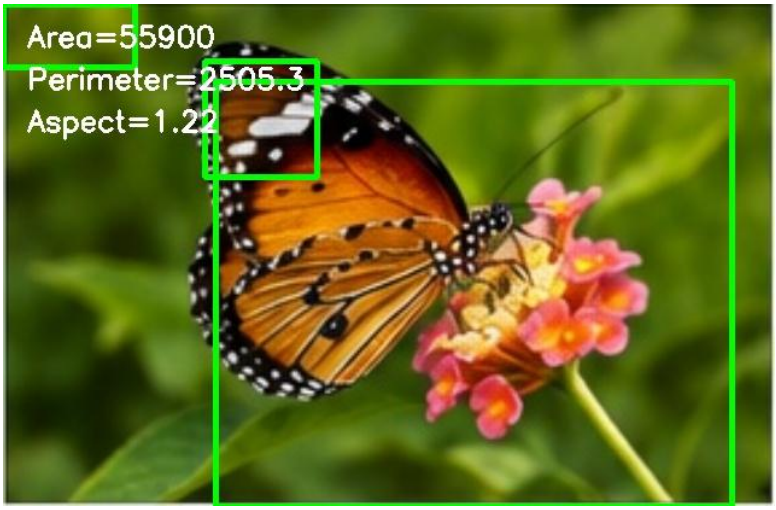
Resized output

Enhanced output

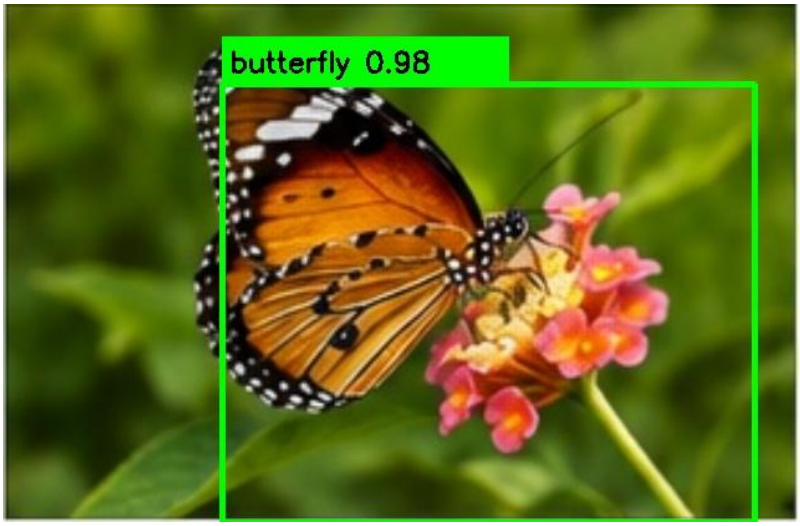


Segmentation output

Edge output



Feature output



Final detection output

Appendix D – Performance Results

Metric	Traditional CV	Compact CNN
Accuracy	95.83%	100.00%
Precision	92.31%	100.00%
Recall	100.00%	100.00%
F1-score	96.00%	100.00%
Latency	0.062 ms	1.467 ms
Estimated FPS	16,035.6	681.9

Appendix E – Important Implementation Note

The quantitative results presented in this report were obtained by executing the prototype workflow using the supplied butterfly image and a controlled augmented evaluation set. Therefore, the reported 100% CNN recognition result represents a demonstration result on a small synthetic/augmented benchmark and must not be interpreted as 100% real-world recognition accuracy.

For reliable academic or industrial deployment, the model should be evaluated using an independent and unseen dataset containing multiple butterfly images with variations in background, pose, orientation, lighting, scale, and image quality. Similarly, for the original industrial inspection problem, evaluation should be performed using a sufficiently large dataset of genuine manufacturing defect images.